✅ What is Query Optimization?

Query optimization means:

Making SQL queries run **faster** and use **less memory and CPU**.

It helps to:

- Reduce execution time ⏱️
- Reduce server load 💻
- Improve database performance 🚀
- Handle large data efficiently 📊

🔥 Main Goal of Query Optimization

✓ Reduce unnecessary data access
✓ Reduce scanning of large tables
✓ Use indexes properly
✓ Improve execution plan

📌 Common Query Optimization Techniques

1️⃣ Avoid Accessing Unnecessary Data

❌ Bad Practice:

```
SELECT * FROM messy_indian_dataset;
```

Why bad?

- Fetches all columns
- Wastes memory
- Slower for large tables

✅ Good Practice:

```
SELECT id, name, city, purchase_amount
FROM messy_indian_dataset;
```

👉 Only required columns are fetched → Faster

✅ Add Filtering:

```
SELECT id, name, city, purchase_amount
FROM messy_indian_dataset
WHERE purchase_amount > 1000
LIMIT 5;
```

👉 Less rows + less columns = Much faster

2️⃣ Use the Right Type of JOIN

Different joins give different results:

Join Type	Use When
INNER JOIN	Only matching rows
LEFT JOIN	All left table rows
RIGHT JOIN	All right table rows
FULL JOIN	All rows from both

Example:

```
SELECT *
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

👉 Use INNER JOIN when you only need matching data (usually faster).

⚠️ Avoid unnecessary joins — they increase execution time.

3️⃣ Use Appropriate Data Types

Correct data types improve:

- Storage efficiency
- Index performance
- Query speed

✅ Good Example:

```
id INT
```

name VARCHAR(50)

description VARCHAR(255)

Use:

- INT → numbers
- VARCHAR → short text
- TEXT → long text

⚠ Don't use TEXT when VARCHAR is enough — it slows indexing.

4️⃣ Use Indexing (Very Important 🔥)

Indexes make searching faster.

Without index:
👉 Database scans entire table (Full Table Scan)

With index:
👉 Database jumps directly to matching rows

Example:

```
CREATE INDEX idx_purchase

ON messy_indian_dataset(purchase_amount);
```

Best columns to index:

- Columns used in WHERE
- Columns used in JOIN
- Columns used in ORDER BY
- Columns used in GROUP BY

⚠ Too many indexes = slower INSERT/UPDATE

5️⃣ Analyze Query Execution Plan

Use EXPLAIN to see how database runs query.

```
EXPLAIN

SELECT id, name, city, purchase_amount

FROM messy_indian_dataset

WHERE purchase_amount > 1000;
```

It shows:

- Index usage
- Table scan type
- Join method
- Estimated rows scanned

👉 If you see “Full Table Scan” → Try adding index.

6️⃣ Use Caching

If query runs frequently and data doesn’t change much:

Use caching to avoid recalculating.

Example (MySQL):

```
SELECT SQL_CACHE id, name, city, purchase_amount
FROM messy_indian_dataset
WHERE purchase_amount > 1000;
```

👉 Useful for reports and dashboards.

7️⃣ Use Temporary Tables

Useful for complex queries.

Instead of repeating heavy calculation:

Step 1:

```
CREATE TEMPORARY TABLE high_purchase AS
SELECT id, name, city, purchase_amount
FROM messy_indian_dataset
WHERE purchase_amount > 1000;
```

Step 2:

```
SELECT *
FROM high_purchase
WHERE city = 'Mumbai';
```

👉 Saves time for repeated operations.

🚀 Extra Optimization Tips (Important for Interviews)

🔷 Use LIMIT when testing

Avoid loading millions of rows while testing.

🔷 Avoid Functions in WHERE

❌ Bad:

```
WHERE YEAR(order_date) = 2024;
```

✅ Good:

```
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

👉 Functions prevent index usage.

🔷 Avoid SELECT DISTINCT unless needed

DISTINCT requires sorting → expensive operation.

🔷 Avoid Subqueries if JOIN works better

Sometimes JOIN performs faster than nested queries.

📊 Optimization Priority Order

Most Important Techniques:

1️⃣ Proper Indexing
2️⃣ Avoid SELECT *
3️⃣ Filter early (WHERE clause)
4️⃣ Use correct JOIN
5️⃣ Analyze EXPLAIN plan

🧠 Easy Memory Trick

Think of optimization like saving fuel:

- Don’t carry extra luggage (avoid extra columns)
- Take shortest route (use indexes)
- Avoid traffic (filter early)
- Don’t repeat trips (use temp tables or caching)

🎯 Final Summary

Query optimization means:

✓ Fetch less data
✓ Filter early
✓ Use indexes
✓ Analyze execution plan
✓ Choose correct joins
✓ Use proper data types